

UNIT 1

Introduction To Database Management System

Basic Definitions

- **Data:** Known facts that can be recorded and have an implicit meaning.
- **Database:** Systematic collection of related data.
- **Database Management System (DBMS):** A collection of software to facilitate the creation and maintenance of a DB.
- **Database System:** The DBMS software together with the data. Sometimes, applications are also included.

Database Example

- **College Database**

- ✓ *Student file*
- ✓ *Course file*
- ✓ *Section file*
- ✓ *Grade-Report file*
- ✓ *Pre-requisite file etc.*

STUDENT

Name	Student number	Class	Major
Smith	17	1	CS
Brown	8	2	CS

COURSE

Course_name	Course_number	Credit_hours	Department
Intro to Computer Science	CS1310	4	CS
Data Structures	CS3320	4	CS
Discrete Mathematics	MATH2410	3	MATH
Database	CS3380	3	CS

SECTION

Section_identifier	Course_number	Semester	Year	Instructor
85	MATH2410	Fall	04	King
92	CS1310	Fall	04	Anderson
102	CS3320	Spring	05	Knuth
112	MATH2410	Fall	05	Cheng
119	CS1310	Fall	05	Anderson
135	CS3380	Fall	05	Stone

GRADE REPORT

Student_number	Section_identifier	Grade
17	112	B
17	119	C
8	85	A
8	92	A
8	102	B
8	135	A

PREREQUISITE

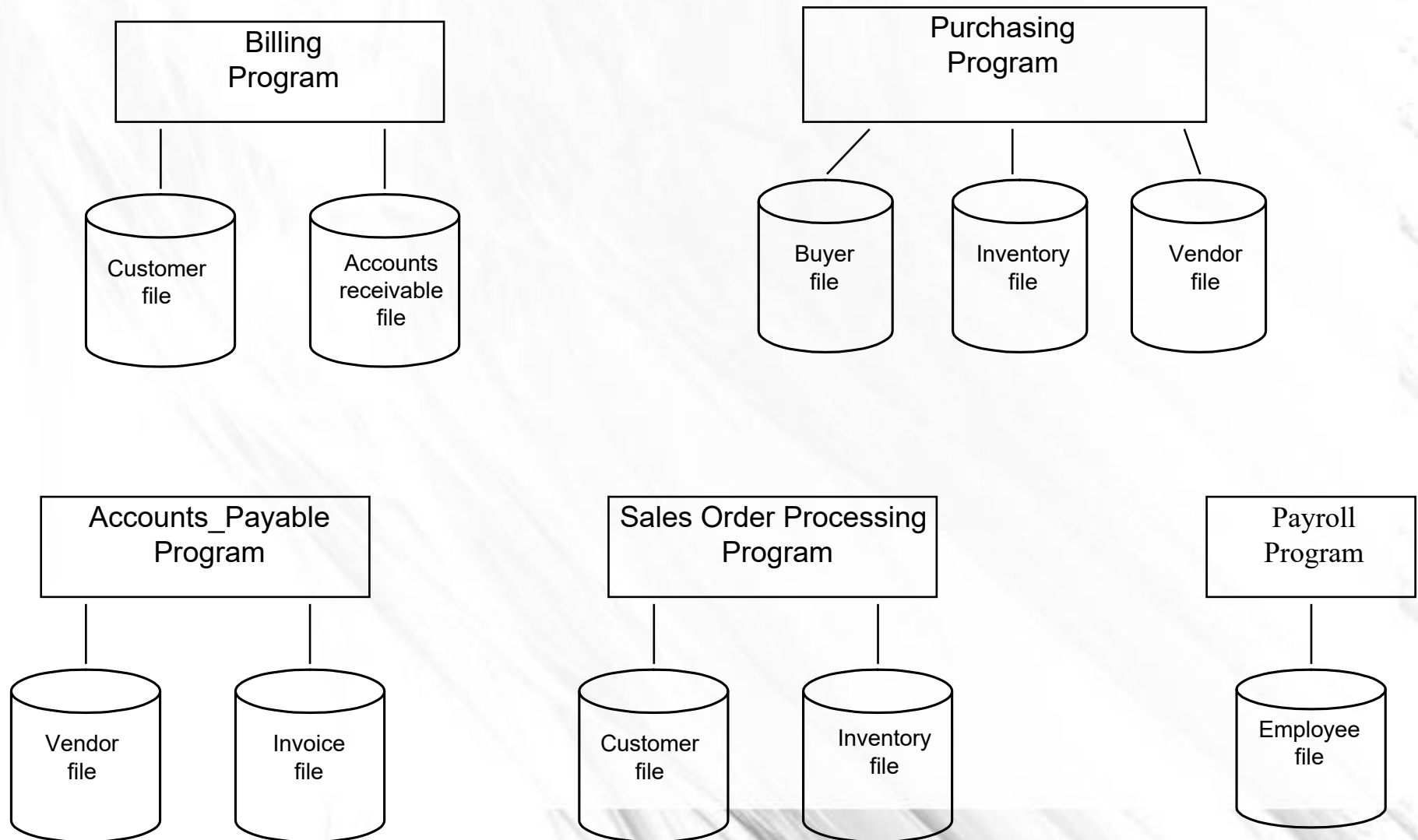
Course_number	Prerequisite_number
CS3380	CS3320
CS3380	MATH2410
CS3320	CS1310



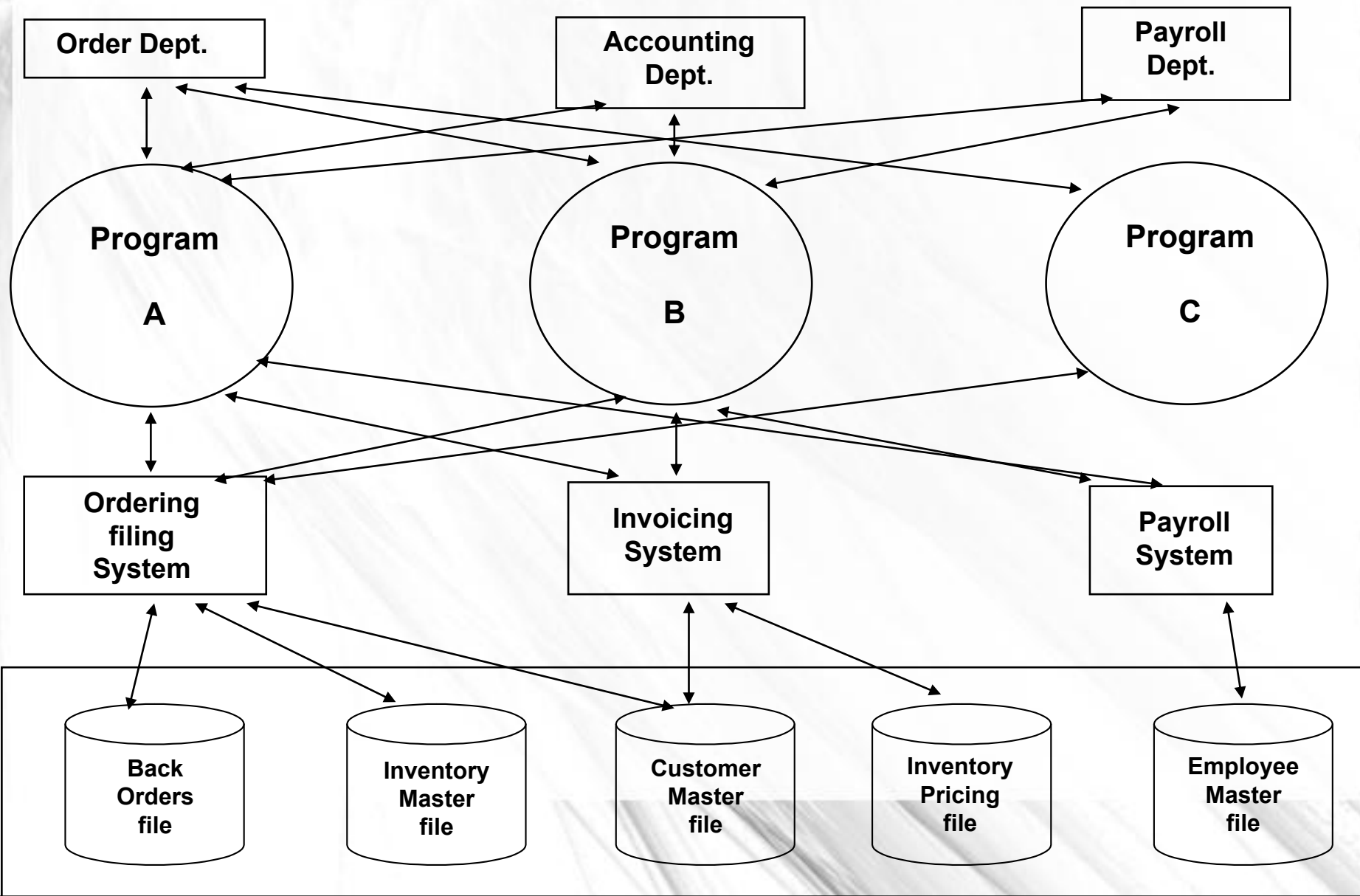
Difference between Traditional File Processing Vs. DBMS



File Processing Systems



Database Approach



Characteristics of Database

- Self-describing nature
- Insulation between program, data and data abstraction
- Support for multiple views of data
- Sharing of data and Multiuser Transaction processing

Actors on the Scene

(job involves day-to-day use of d/b)

- Database Administrator
- Database Designer
- End User
- System Analysts and Application Programmers

Workers Behind the Scene

(People responsible for development & operation of DBMS)

- DBMS system designers & implementers
- Tool Developers
 - ✓ For performance monitoring (SQL Server Profiler)
 - ✓ Natural language or graphical interface (SQL Lite D/b browser)
- Operators & maintenance

Advantages of DBMS

- ✓ Controlling Redundancy
- ✓ Restricting unauthorized access
- ✓ Providing storage structures for efficient query processing
- ✓ Query optimization
- ✓ Providing backup & recovery
- ✓ Representing complex relations and data
- ✓ Enforcing integrity constraints

Advantages (Continued..)

- ✓ Permitting Inferencing and actions using rules
- ✓ Additional advantages
 - Potential for enforcing standards
 - Reduced application development time
 - Flexibility
 - Availability of up-to-date information
 - Economies of scale

DBMS Vs. File System

Difference Factor	File System	DBMS
Definition	Is an abstraction to store, retrieve, manage and update a set of files	A collection of interrelated data and a set of programs to access it
Data Redundancy	Each user defines and implements needed file for a specific application	A single repository of data is maintained that is defined once and accessed by many users
Sharing of data	Doesn't allow sharing of data or data sharing is very complex	Data can be shared easily due to centralized storage
Data consistency	When data is redundant it is difficult to update	Less data redundancy so data remains consistent
Difficult to search/access data	It becomes difficult as need to refer multiple files	Very easy and user friendly. Query operations are already available

DBMS Vs. File System

Difference Factor	File System	DBMS
Data Isolation	No standard format of data, data is scattered in various files	Due to centralized system, format of similar type of data remains same
Data Integrity	Data in d/b must follow or satisfy some rules or constraints to avoid invalid data entry	Maintains data integrity by providing constraints
Security problems	No or less security. Majorly provided are locks, guards	High level of security as encryption, passwords
Concurrent access anomalies	File system doesn't provide any remedy for it	Provides functionality for concurrent access of transactions

A Brief History D/B Applications

- ✓ Early D/b applications using hierarchical & network systems
- ✓ Providing application flexibility with relational databases
- ✓ Object oriented applications & the need for more complex databases
- ✓ Interchanging data on the web for E-commerce
- ✓ Extending database capabilities for new applications

Hierarchical System

- ✓ Uses one-to-many relationship
- ✓ Presents data in tree-like structure
- ✓ E.g. IBM's IMS(Information management system), .xml file

01 LIBRARY-SEGMENT.

05 BOOK-ID PIC X(5).

05 ISSUE-DATE PIC X(10).

05 RETURN-DATE PIC X(10).

05 STUDENT-ID PIC A(25).

01 BOOK-SEGMENT.

05 BOOK-ID PIC X(5).

05 BOOK-NAME PIC A(30).

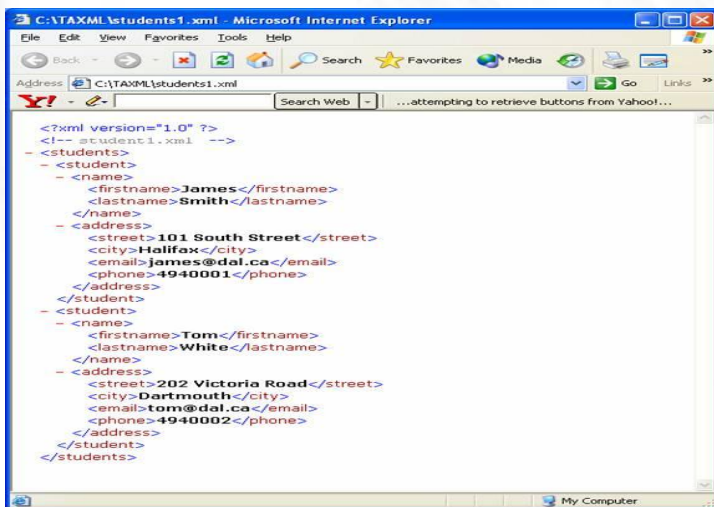
05 AUTHOR PIC A(25).

01 STUDENT-SEGMENT.

05 STUDENT-ID PIC X(5).

05 STUDENT-NAME PIC A(25).

05 DIVISION PIC X(10).



Disadvantages (When not to use DBMS)

- ✓ Huge investment in s/w, h/w & training
- ✓ Overhead for providing security, concurrency control, recovery, and integrity functions
- ✓ Simple, well defined, not expected to change applications
- ✓ Multiple user access to data is not required

Contents

- Entity types, Entity sets, Attributes, Keys
- Relationship types, Relationship sets, roles, Structural Constraints
- Strong and weak entity
- E-R diagram
- Relationship types of degree higher than 2
- Enhanced ER model
- **UML (Unified Modelling Language)**

E-R Model

- High level conceptual data model
- No implementation details
- Used to communicate to non technical users
- Describes the data as entities, relationships and attributes.

Database application - “COMPANY”

1. Company is organized into departments. Each department has unique name, number and particular employee who manages it. We keep track of the start date when that employee began managing the department. A department may have several locations.



Requirements

- 2. Department controls a number of projects, each of which has a unique name, a unique number and a single location.**
- 3. We need to store each employee's name, number, address, salary, gender, and date of birth. An employee is assigned to one department but may work on several projects, which are not necessarily controlled by the same department.**



Requirements

4. We want to keep track of the dependents of each employee for insurance purposes. We keep each dependent's first name, Gender, date of birth and relationship to employee



Requirements

5. We keep track of the number of hours per week that an employee works on each project. We also keep track of the direct supervisor of each employee



Requirements



phanmemkhomienphi.com

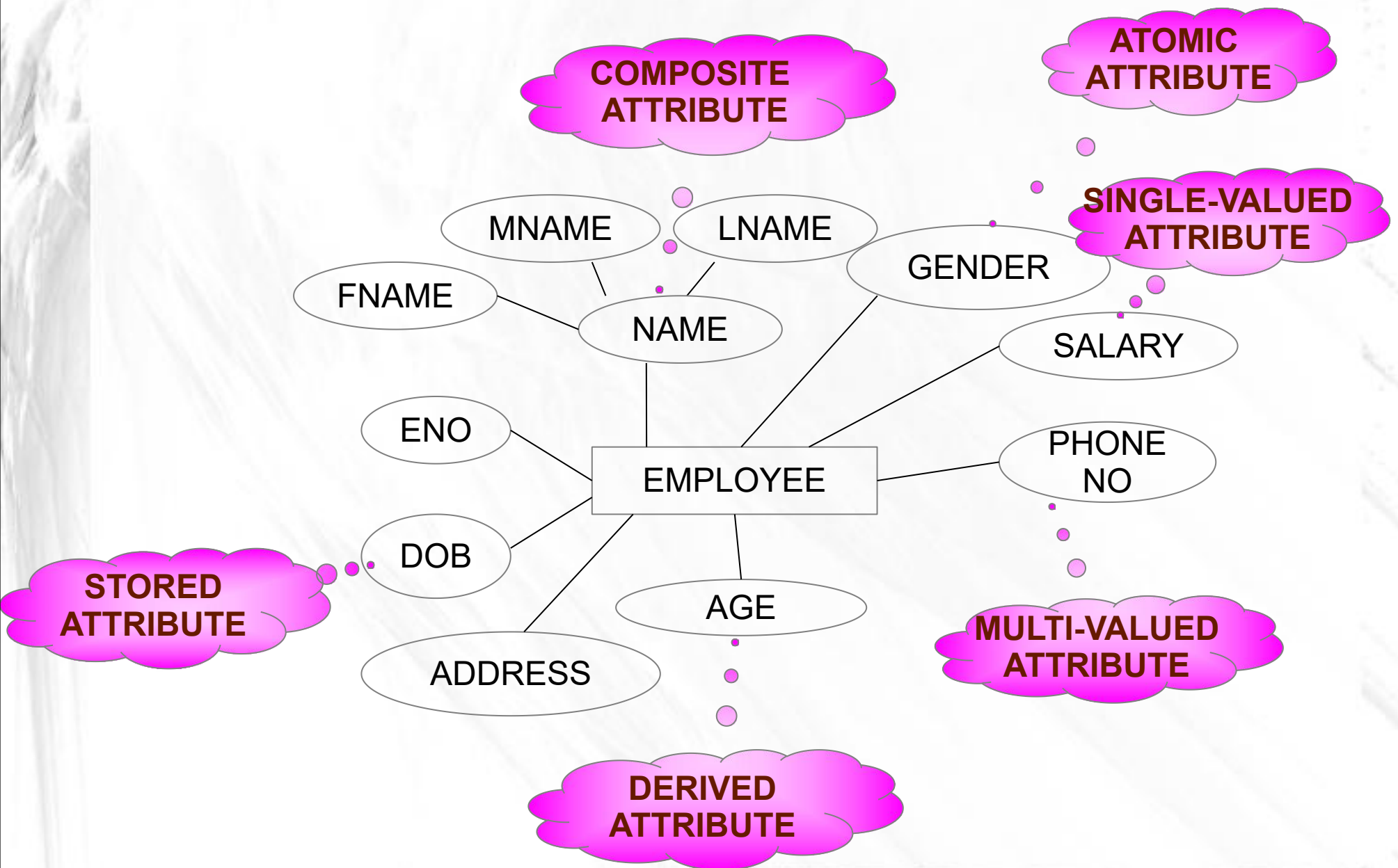
Basic Concepts

- **Entity:**

- Thing in the real world with an independent existence.
- e.g. A particular person, a car
A Company, a job, a university course

- **Attributes:**

- Particular properties that describe an entity



Composite Vs Atomic/Simple Attributes

Atomic Attributes : Attributes that are not divisible.

- ✓ Value of composite attribute is concatenation of the values of constituent atomic attributes

Example Pin Number

Composite Attributes : Can be divided into smaller sub parts which represent more basic attributes with independent meanings

Example Fullname

Single Valued Vs Multi-Valued Attributes

Single Valued Attributes : Attributes having single value for a particular entity. E.g. salary, DOB etc.

Multi-Valued Attributes : Attributes having a set of values for a particular entity. E.g. phone number, color of a car

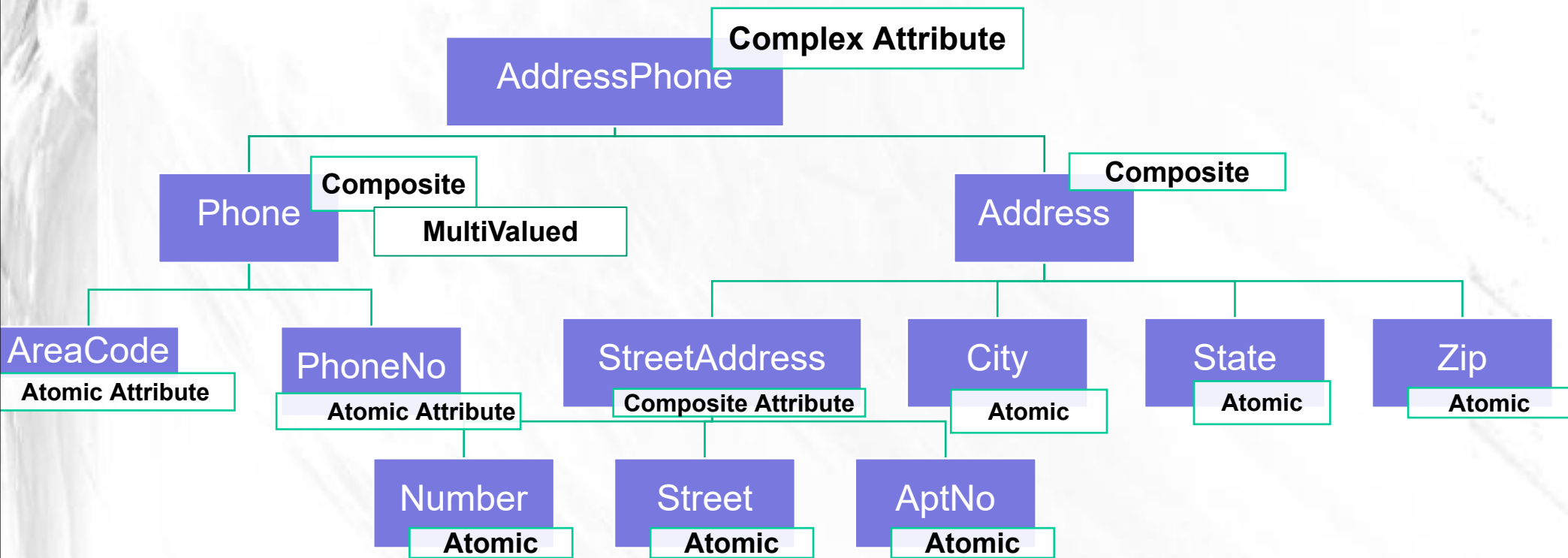
Stored Vs Derived Attributes

- ✓ Two or more attribute values are related.
- ✓ **E.g.** age and DOB of a person.
- ✓ For a person age of a person can be derived from DOB of a person

Complex Attributes

- ✓ A group of composite and multivalued attributes
- ✓ Composite attributes represented using () where every atomic attribute separated by ,
- ✓ Multivalued attributes represented using { }
- ✓ **E.g.** AddressPhone for a person

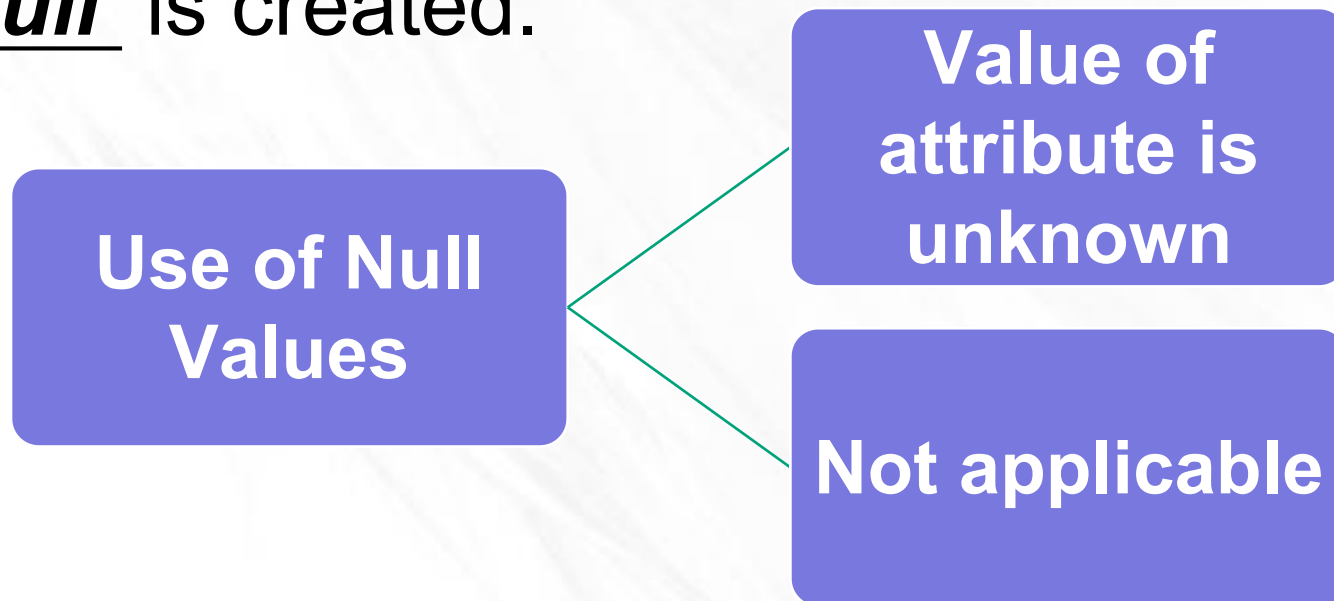
{AddressPhone({Phone (AreaCode, PhoneNo)} ,
Address(StreetAddress(Number, Street,
AptNo), City, State, Zip)) }



{AddressPhone({Phone (**AreaCode**, **PhoneNo**)} ,
 Address(StreetAddress(**Number**, **Street**, **AptNo**),
 City, State, Zip)) }

Null Values

- ✓ An entity doesn't have an applicable value for some attribute then special value called **'Null'** is created.



Employment Application

First name *

Last name *

Email *

Portfolio website

Position you are applying for *

Salary requirements

When can you start?

Phone *

Fax

Are you willing to relocate?

☒ Yes ☐ No ☐ Not sure

Last company you worked for

Reference / Comments / Questions

Enter security code

SECURITY CODE

8YM28

[Reload Image](#)

[Send Application](#)

Entity Type

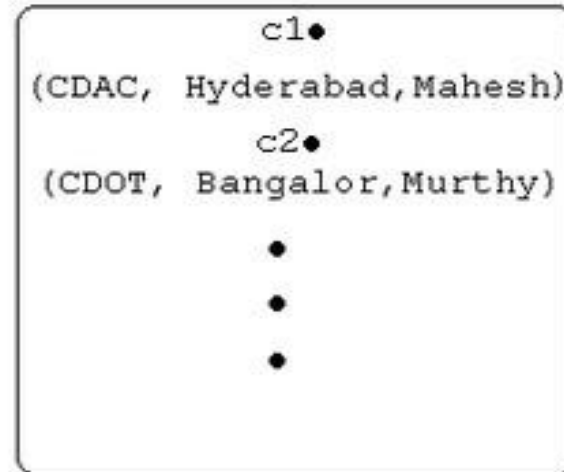
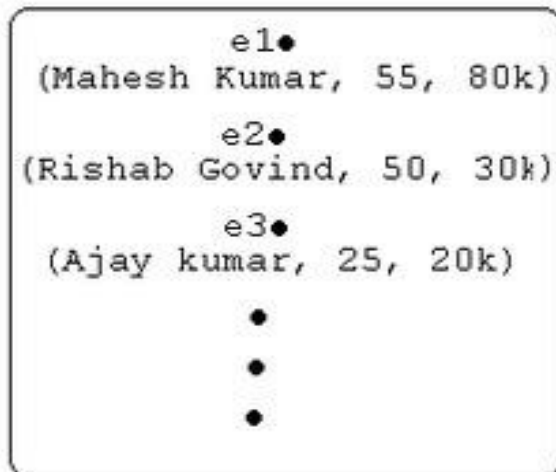
- ✓ A collection (or set) of entities that have same attributes
- ✓ Is described by its name and attributes.

ENTITY TYPE NAME:
ATTRIBUTES:

EMPLOYEE
Name, age, salary

COMPANY
Name, Headquarters,
president

ENTITY SET:
(EXTENSION)



Entity Set

- ✓ Collection of all entities of a particular entity type at the particular moment of a time
- ✓ Usually referred using the same name as the entity name

Key Attribute

- ✓ Important constraint on entities of an entity type
- ✓ Uniqueness constraint
- ✓ **E.g.** employee_name for an Employee entity type

Value Sets (Domains) of Attributes

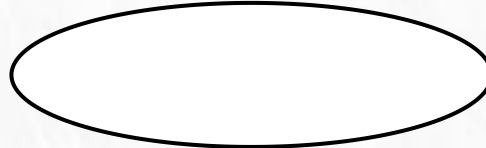
- ✓ Value assigned to attribute of an individual entity is selected from Value sets or Domains of Attributes.
- ✓ **E.g.** Age of an employee
- ✓ Value sets are typically specified using the basic data types e.g. integer, date, time etc.

Notations

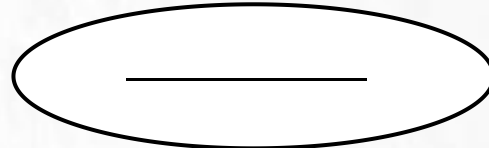
- Entity Type



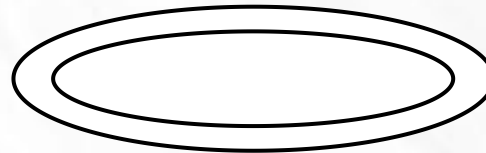
- Attribute



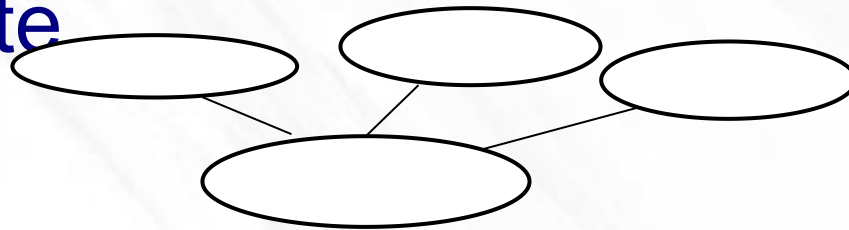
- Key Attribute



- Multi Valued
attribute



- Composite
attribute

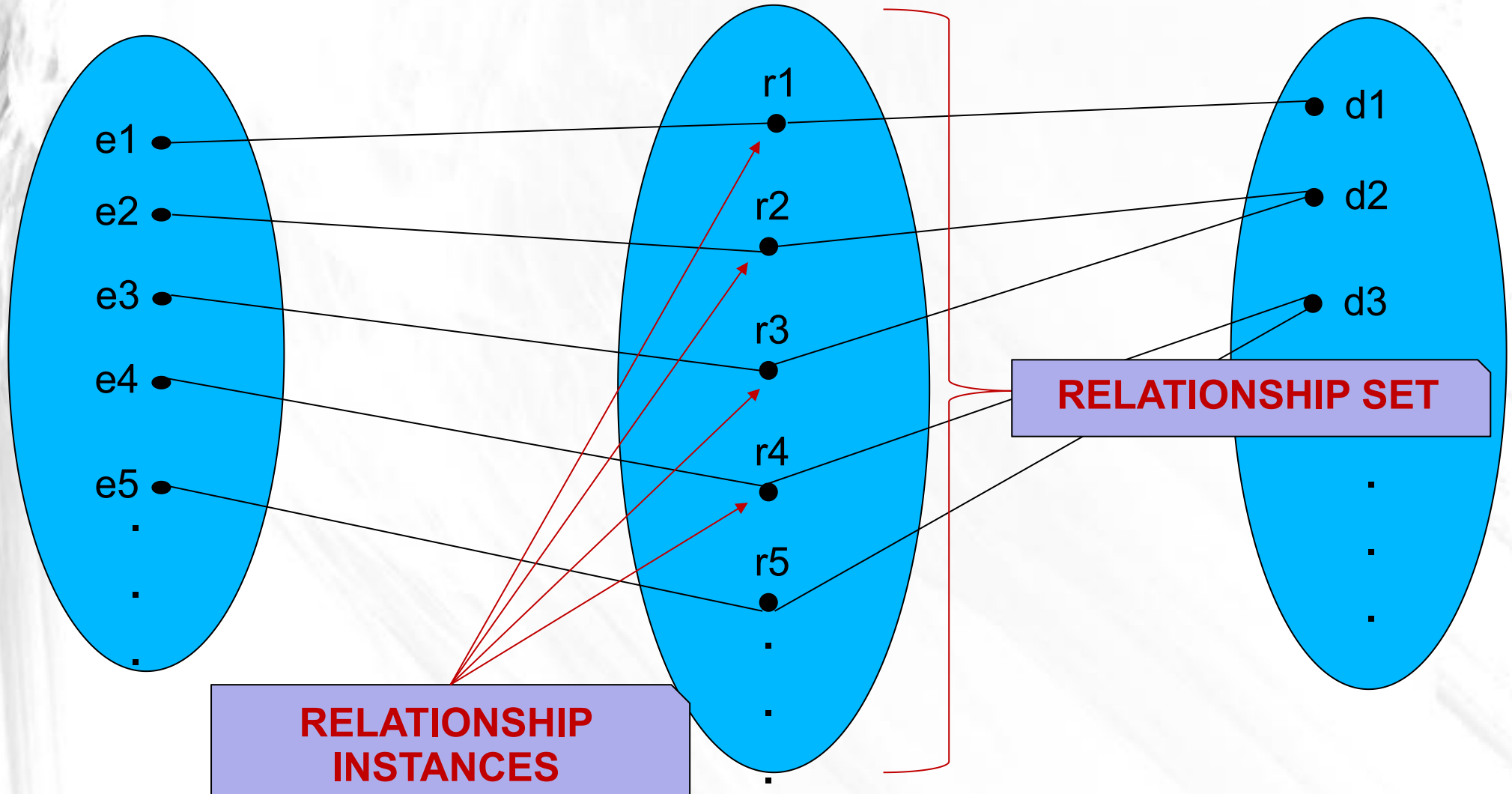


RELATIONSHIP TYPE

EMPLOYEE

WORKS_FOR

DEPARTMENT



Relationship Type, Set and Instances

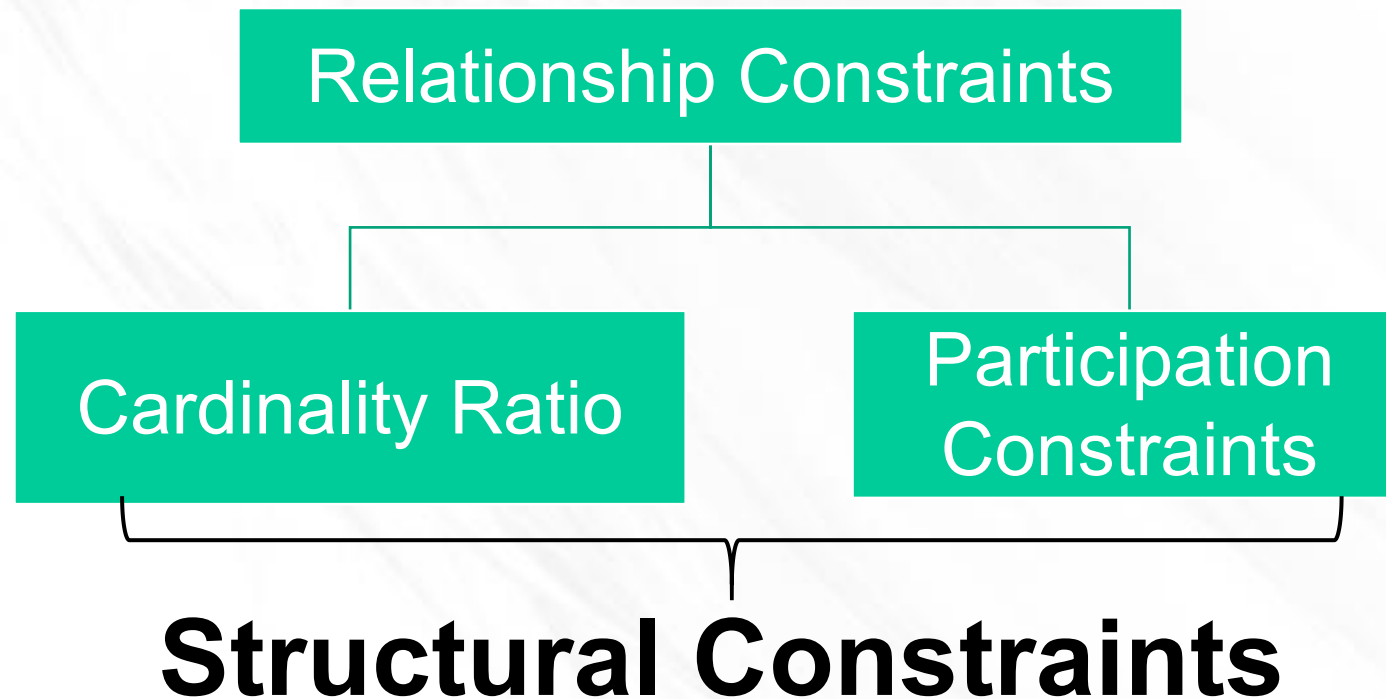
- ❖ **Relationship Type 'R'** among entity types $E_1, E_2, \dots, E_n$ defines a set of associations among entities from these entity types
- ❖ **Relationship set 'R'** is a set of relationship instances r_p where each r_i (from r_p) associates n individual entities $(e_1, e_2, \dots, e_n)$ and each entity e_j in r_i is belong to Entity type E_j , $1 \leq j \leq n$

Relationship Degree

- ✓ Number of participating entity types
- ✓ WORKS_FOR relationship degree = **2, Binary**
- ✓ Relationship degree = **3, Ternary**

Constraints on Relationship Types

- ✓ Which limit the possible combinations of entities that may participate in the corresponding relationship set



Cardinality Ratio

- ✓ Number of maximum relationship instances that an entity can participate in.

Entity Types	Relationship Type	Cardinality Ratio
DEPARTMENT : EMPLOYEE	WORK_ON	N : 1
EMPLOYEE : DEPARTMENT	MANAGES	1 : 1
EMPLOYEE : PROJECT	WORKS_ON	M : N

Participation Constraints and Existence Dependencies

- ✓ Participation constraint specifies whether the existence of an entity depends on whether it is related to another entity using the relationship type
- ✓ Also called as Minimum cardinality constraint

Types of Participation Constraints

- ✓ Total : Every entity from one entity type must be related to another entity type via relationship type
e.g Every employee must work for a department. So, employee entity exists iff it participates in at least one WORKS_ON relationship instance
- ✓ Also called as Existence Dependency

✓ Partial : Some of entities from one entity type are related to another entity type via relationship type

e.g Not every employee manages the department so, participation of EMPLOYEE in the MANAGES relationship type is partial

Identifying Relationship Types

Relationship Type	Cardinality Ratio	Participating entities
MANAGES	1:1	EMPLOYEE : DEPARTMENT
WORKS_FOR	1 : N	DEPARTMENT : EMPLOYEE
CONTROLS	1 : N	DEPARTMENT : PROJECT
SUPERVISION	1 : N	EMPLOYEE(SUPERVISOR) : EMPLOYEE(SUPERVISEE)
WORKS_ON	M : N	EMPLOYEE : PROJECT
DEPENDENTS_OF	1 : N	EMPLOYEE : DEPENDENT

